QUT IT Capstone | Team 115

Student Attrition Risk & Intervention Briefing

Technical Documentation

Project: P965 – Databricks AI-Driven Insight Discovery for Queensland University of Technology

Industry partner: QUT Digital Business Solutions

User story: US-22 Technical and User Documentation (GitHub issue #29) – technical documentation component

Repository: https://github.com/karen-lee1029/Capstone_T115 (branch `main`)

Evidence baseline: repository commit `00c6818` (14 October 2026); application source last changed in commit `385041c`

Document status: Draft for team review. Not yet reviewed or approved by the team or the industry partner.

> **Companion document.** Advisor-facing instructions are covered by the separate user documentation (for example, T115 Application Documentation Review v2). This document covers architecture, setup, deployment, operation, maintenance and limitations.

Contents

1. Document overview 4
1.1 Purpose 4
1.2 Audience 4
1.3 Scope 4
1.4 How evidence is labelled 4
1.5 Sources inspected 4
2. Solution overview 5
2.1 Components 5
2.2 End-to-end workflow 5
3. Architecture 6
3.1 Logical architecture 6
3.2 Application structure 6
3.3 Technology stack 7
4. Data layer 8
4.1 Unity Catalog objects 8
4.2 Synthetic data 8
4.3 Prediction table schema 8
4.4 Student identifiers 9
4.5 Small-group suppression 9
5. Machine-learning component 10
5.1 Target and modelling grain 10
5.2 The 21 approved features 10
5.3 Training pipeline 10
5.4 Re-running the model 11
5.5 How the score must be read 11
6. Student Attrition Risk Overview dashboard 12
6.1 Datasets 12
6.2 Pages and widgets 12
6.3 Deploying or updating the dashboard 12
7. Application interfaces 13
7.1 Student Advisor Briefing page (/ui) 13
7.2 REST API (/api) 13
7.3 MCP server (/mcp) 13
8. Advisor Briefing workflow 13
8.1 Orchestration 14
8.2 Prompt 14
8.3 Validation rules 14
8.4 Storage 15
8.5 Logging 15
9. Error handling 15
10. Setup and configuration 17
10.1 Prerequisites 17
10.2 Configuration reference 17

10.3 Local run (mock mode) 17
10.4 Local run against Databricks 18
10.5 Deploying the Databricks App 18
11. Operation and maintenance 18
12. Testing and quality 19
13. Security and privacy 19
13.1 Access control 19
13.2 Data protection 19
14. Known limitations and open defects 20
14.1 Solution limitations 20
14.2 Open defects accepted at delivery 20
14.3 Documentation drift 20
15. Items requiring team confirmation 21
16. Verification checklist 22
17. US-22 acceptance review (technical documentation) 23
Document history 23

1. Document overview

1.1 Purpose

This document describes how the Team 115 solution is built, configured, deployed, operated and maintained. It is written so that QUT Digital Business Solutions (DBS) staff can understand and support the solution after the capstone team hands it over, without needing the development history.

1.2 Audience

- **Primary:** DBS technical staff who will deploy, operate or maintain the solution in a Databricks workspace.
- **Secondary:** future developers, and reviewers assessing US-22 acceptance.

Readers are assumed to be familiar with Databricks workspaces, Unity Catalog, SQL warehouses and Python projects.

1.3 Scope

In scope: the synthetic data and machine-learning (ML) notebooks, the prediction Delta table, the AI/BI dashboard, the Databricks App (web page, REST API and MCP server), the AI-assisted Advisor Briefing workflow, configuration, deployment, testing, security, privacy and known limitations.

Out of scope: step-by-step advisor instructions (see the user documentation), and project-management records such as sprint history.

1.4 How evidence is labelled

Every implementation-specific statement was checked against the repository. Where a statement could not be checked, it is labelled. The labels used in this document and in the verification checklist (Section 16) are:

Label	Meaning
Verified – code	Confirmed by reading the source, configuration or dashboard definition at the evidence baseline.
Verified – run	Confirmed by running the test suite or the application locally in mock mode on 16 October 2026.
Documented	Stated in a project record (specification, defect register, notebook notes) but not re-executed for this document.
Requires confirmation	Depends on the Databricks workspace or a team decision that could not be inspected. Listed in Section 15.

1.5 Sources inspected

- Application source, tests and configuration in student_attrition_risk_app.
- Dashboard definition Student Attrition Risk Overview.lvdash.json.
- Notebooks in notebooks/ and the ML specification set in docs/Machine Learning Model Development Documentation.
- Feature specifications in `specs/001` to `specs/006`, including the final defect register.
- The Advisor Briefing acceptance criteria and system prompt document.
- GitHub issues #29 US-22, #19 US-12, #20 US-13, #23 US-16, #26 US-19 and #28 US-21.

> **Access limitation.** Issues were read through their public web pages only; the project board, API and comments were not accessible. Every inspected issue still shows "Planned", so issue status was not used as evidence of completion – the code was. The Databricks workspace (published dashboard, deployed app, endpoint, Volume, permissions) was not accessible.

2. Solution overview

The solution is a proof of concept that helps academic advisors review model-generated student attrition-risk signals and prepare a supportive, human-led response. It runs entirely inside a Databricks workspace and uses **synthetic, de-identified student data** – no real student records are used.

2.1 Components

Component	What it does	Where it lives
Synthetic data generation	Builds a synthetic student-enrolment fact table and dimension tables that mirror the industry partner's schema.	DBLDatagen Synthetic Data Generation V2.ipynb
ML attrition model	Trains a Spark ML Random Forest, logs it to MLflow and scores every student into a prediction Delta table.	Machine Learning Model.ipynb
Student Attrition Risk Overview dashboard	Databricks AI/BI (Lakeview) dashboard for aggregate risk patterns, filtering and finding a de-identified Student ID.	dashboard/…lvdash.json
Student Advisor Briefing app	Databricks App serving a Streamlit page, a REST API and an MCP server over one shared service layer.	student_attrition_risk_app/src/student_attrition_risk
AI-assisted Advisor Briefing	Generates a four-section briefing through a Databricks Model Serving endpoint, validates it, retries once if needed and stores the validated result in a Unity Catalog Volume.	`student_service.py`, `briefing_*.py`, `retry_workflow.py`

2.2 End-to-end workflow

1. The synthetic data notebook writes the source Delta tables to `workspace.student_aggregate`.
2. The ML notebook trains the model and overwrites `student_attrition_risk_prediction` with one risk result per student.
3. The advisor uses the dashboard to explore aggregate patterns and copy a 16-character de-identified Student ID.
4. The advisor opens the app, enters the Student ID and selects **Retrieve**. The app links to the dashboard (**View Dashboard**), but the dashboard has no link back to the app, so users need the app URL separately.
5. For a student flagged At Risk, **Generate Advisor Briefing** returns the saved validated briefing if one exists, or generates, validates and saves a new one. **Regenerate** forces a new one.
6. The advisor reviews the AI-generated briefing and can download it as a PDF. The advisor, not the system, decides whether any support or intervention is appropriate.

> **Decision-support boundary.** The application never decides that an intervention must occur. The risk score is a relative ranking score, not a probability (Section 5.5), and the briefing is AI-generated decision support that must be reviewed by a person.

3. Architecture

3.1 Logical architecture

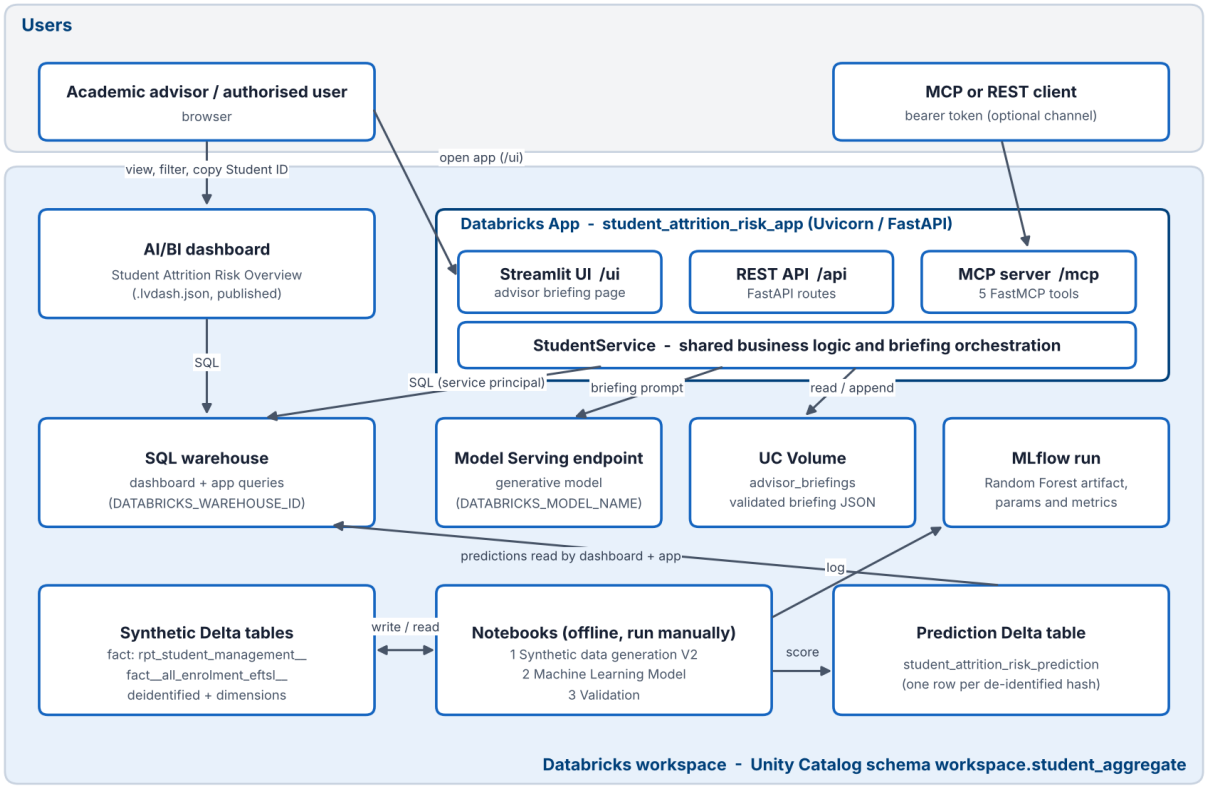


Figure 1. Solution architecture (drawn from the repository code and configuration; workspace resource names in the app configuration require confirmation for a new workspace).

3.2 Application structure

The app is a single Python process. `main.py` builds one FastAPI application and mounts three interfaces on it. All three call the same `StudentService`, so business rules (for example, "only at-risk students can receive a briefing") are enforced in one place. Data access, generation, validation, retry and storage sit behind small interfaces ("seams") defined in `ports.py`, which is why the tests can replace them with in-memory versions.

```
Streamlit UI (/ui)  ┐
REST API     (/api) ├─> StudentService -> StudentRepository -> SQL warehouse
MCP server   (/mcp) ┘                 │
                                      └─> Instructions → GenerationProvider → Validator
                                          → RetryWorkflow → BriefingStore (UC Volume)
```

Module	Responsibility
`main.py`	Composition root. Chooses mock or Databricks repository, real or stub generation provider, and Volume or in-memory store from configuration. Starts Streamlit as a child process and mounts `/ui`, `/api` and `/mcp`. `/` redirects to `/ui/`. If configuration is invalid, serves a 503 on every route naming the problem.
`config.py`	Reads environment variables into `Settings`; validates three-part table names and the Volume path.
`databricks_client.py`	Opens a Databricks SQL connection using Databricks unified authentication, or an explicitly supplied `DATABRICKS_TOKEN`.
`student_repository.py`	SQL access: prediction lookup by full hash or 16-character Student ID, snapshot fields, the 21 model features, high-risk list, health check. Also `MockStudentRepository` (three synthetic records).

Module	Responsibility
`student_service.py`	Business logic and briefing orchestration; maps failures to typed errors; writes metadata-only workflow log lines.
`briefing_instructions.py`	Composes the generation prompt (`StructuredBriefingInstructions`, id `structured-advisor-v2`).
`briefing_provider.py`	Calls the Model Serving endpoint (`DatabricksGenerationProvider`); `StubGenerationProvider` fails explicitly when no endpoint is configured.
`briefing_validation.py`	Rule-based checks of a draft against the briefing acceptance criteria (`StructuredBriefingValidator`, id `structured-validator`).
`retry_workflow.py`	Exactly one additional attempt after a failed first attempt (`SingleRetryWorkflow`).
`briefing_store.py`	Append-only JSON storage in a Unity Catalog Volume (`VolumeBriefingStore`); `InMemoryBriefingStore` for local use and tests.
`briefing_messages.py`	Advisor-facing wording for save and retrieval notices.
`briefing_pdf.py`	Builds the downloadable PDF briefing with ReportLab.
`ui.py`, `streamlit_host.py`	The Streamlit page, and the process host plus HTTP/WebSocket proxy that serves it under `/ui`.
`api.py`, `mcp_server.py`	REST routes and FastMCP tools (Section 7).

3.3 Technology stack

Layer	Technology (from repository)
Platform	Databricks: Unity Catalog, Delta Lake, serverless notebooks, SQL warehouse, AI/BI dashboard, Databricks Apps, Model Serving, MLflow
Data generation	`dbldatagen` (Databricks Labs Data Generator), PySpark
Machine learning	Spark ML `RandomForestClassifier`, MLflow
Application	Python 3.11–3.13, FastAPI, Uvicorn, Streamlit, FastMCP 2.x, Pydantic 2, Databricks SDK, Databricks SQL Connector, ReportLab, httpx, websockets
Tooling	`uv` with `uv.lock`, pytest, Ruff

4. Data layer

4.1 Unity Catalog objects

All objects are in catalog `workspace`, schema `student_aggregate` (from app.yaml, config.py and the notebooks).

Object	Type	Written by	Read by
`rpt_student_management__fact__all_enrolment_eftsl__deidentified`	Delta table (fact)	Data generation notebook	ML notebook, dashboard, app
`dwh_curriculum__course`	Delta table (dimension)	Data generation notebook	ML notebook, dashboard, app
`dwh_learning_and_teaching__teaching_period`	Delta table (dimension)	Data generation notebook	ML notebook, app
`dwh_curriculum__study_area_b`	Delta table (dimension)	Data generation notebook	Dashboard (Student Enrolment Details dataset)
Other generated dimensions (course offering, unit, module, organisation, etc.)	Delta tables	Data generation notebook	Profiled by ML notebook; not used by the app
`student_attrition_risk_prediction`	Delta table (output)	ML notebook (overwrite)	Dashboard, app
`advisor_briefings`	UC Volume	App (`VolumeBriefingStore`)	App
`mlflow_tmp`	UC Volume	ML notebook (staging only)	MLflow model logging

4.2 Synthetic data

The data generation notebook builds synthetic rows with `dbldatagen`, drawing simple categorical fields from category proportions and conditional fields (EFTSL, credit points, commencing/continuing) from distributions supplied by the industry partner (the CSV files under `notebooks/Data Generation V2/curated`). It writes every table in **overwrite** mode, so re-running it regenerates the whole dataset. Validation evidence is in docs/T115 Synthetic Data Validation.

Each student has exactly one fact row (documented: 973,770 rows and 973,770 distinct students in the 6 August 2026 run). The data is a single cross-sectional snapshot, so it cannot support statements about change over time.

4.3 Prediction table schema

One row per `student_deidentified_hash` (Verified – code: columns read by `student_repository.py` and written in notebook Section 25).

Column	Meaning
`student_deidentified_hash`	De-identified student key; join key to the fact table.
`attrition_risk_percentage`	Positive-class model output × 100 (0–100). A relative ranking score, not a calibrated probability.
`attrition_risk_flag`	True when the unrounded model output ≥ `prediction_threshold`.
`prediction_threshold`	0.50 (fixed in the ML specification).
`mlflow_run_id`	MLflow run that produced the model.

Column	Meaning
`scored_at`	UTC timestamp of the scoring run.

4.4 Student identifiers

- The full identifier is the de-identified hash. The dashboard shows only its first 16 characters as **Student ID (de-identified)** (`LEFT(hash, 16)`).
- The app's profile lookup accepts either the full hash or the 16-character ID. If a 16-character ID matches more than one student, it is treated as not found (Verified – code, `StudentService.get_student_profile`).
- The REST and MCP briefing operations accept only the full hash; the web page passes the full hash after a successful lookup.

4.5 Small-group suppression

When the app displays snapshot fields, values of protected fields (age band, socioeconomic status, regional/remote status, gender, international/domestic, First Nations status and course fields) that represent **fewer than six students** are replaced with "Suppressed for privacy" (`_mask_small_groups`). This masking is **not** applied to the 21 feature values sent to the generative model; that line is deliberately commented out and recorded as design decision DD-2 in the defect register.

5. Machine-learning component

Source: Machine Learning Model.ipynb and the ML plan. The notebook was approved in three human-gated phases on 6 August 2026 (Documented). It is checked in without cell outputs, so metric values are not reproduced here.

5.1 Target and modelling grain

- Target: `is_twelve_month_student_attrition` – whether a student leaves university study within twelve months (not course attrition).
- `is_twelve_month_course_attrition` is excluded to prevent target leakage.
- Grain: one row per `student_deidentified_hash`, assembled from the fact table joined to the course and teaching-period dimensions.

5.2 The 21 approved features

Source	Features
Fact table (16)	`age_at_census`, `socioeconomic_status`, `regional_remote_status`, `student_gender`, `student_is_international_student`, `student_is_first_nations_student`, `attendance_mode`, `eftsl`, `commencing_continuing`, `commencing_continuing_period`, `course_admission_load_category`, `enrolment_year`, `cumulative_credit_points_enrolled`, `cumulative_credit_points_passed`, `cumulative_credit_points_failed`, `cumulative_credit_points_withdrawn`
Course dimension (4)	`course_group`, `broad_primary_field_of_education`, `narrow_primary_field_of_education`, `detailed_primary_field_of_education`
Teaching-period dimension (1)	`teaching_period`

The same list is hard-coded in the app (`MODEL_FEATURE_COLUMNS` in `student_repository.py`) and is the only feature context passed to the generative model.

5.3 Training pipeline

Step	Approved approach
Split	Deterministic 70/15/15 train/validation/test, by hashing the student hash with seed 42 into 100 buckets.
Pre-processing	Training-split medians for missing numeric values; manual one-hot encoding with a bucket for null/unseen categories; `VectorAssembler`. No scaling.
Class imbalance	Balanced class weights from the training split when the positive class is under 40% (base rate about 5.5%).
Model	`RandomForestClassifier`: 100 trees, max depth 7, min 20 instances per node, `sqrt` feature subset, seed 42. Reduced from 300/10/5 to fit Databricks Free Edition serverless memory limits.
Evaluation	Recall (positive class) as primary metric; precision, F1, accuracy, ROC-AUC and confusion matrix on validation and test.
Tracking	MLflow run with parameters, metrics and the model artifact; `mlflow_tmp` Volume used for staging.
Scoring	Every valid student row is scored; the result is validated in a staging table, published to `student_attrition_risk_prediction` in overwrite mode, and the staging table is dropped.

5.4 Re-running the model

1. Open the notebook in the workspace on serverless compute and run the **Session reset** cell (Section 0).
2. Run all cells from the top. Each phase stops on a failed assertion rather than writing partial output.
3. Check the Phase 2 and Phase 3 summary cells and the post-publish verification (Section 29).
4. The dashboard and app read the new predictions immediately; no app redeployment is needed. Existing saved briefings are **not** regenerated and may describe the previous score (Section 14).

No scheduled job, retraining or drift monitoring exists in the repository; these were out of scope.

5.5 How the score must be read

The ML plan records three limitations (L-1 to L-3) that govern every downstream use:

- **L-1:** in the synthetic generator, field of education seen by the model is decoupled from the value that generated the target.
- **L-2:** all 21 features are close to flat against the target. ROC-AUC near 0.5 and precision near the 5.5% base rate are the expected result for this data.
- **L-3:** because class weights recentre outputs near 0.50, `attrition_risk_percentage` is a **relative ranking score, not a likelihood**. A score of 52 does not mean a 52% chance of leaving.

Global feature importance from the notebook describes the model overall. No per-student explanation (for example, SHAP values) is produced, so no attribute may be presented as the reason for an individual student's score.

6. Student Attrition Risk Overview dashboard

A Databricks AI/BI dashboard defined in Student Attrition Risk Overview.lvdash.json (current layout from the US-27 update, commit `3fc0a9f`). It is separate from the app: it runs on the SQL warehouse under Databricks dashboard sharing and is linked from the app by a **View Dashboard** button.

6.1 Datasets

Dataset	Definition
student_attrition_risk_prediction	Prediction table left-joined to each student's latest fact row (by census date, then enrolment year) and the course dimension. Derived fields: **Risk Level** (`At Risk` when percentage ≥ 50, else `Not At Risk`), **Risk Score Range** (eight buckets from 0–46% to 54–100%), **Student ID (de-identified)** (`LEFT(hash, 16)`), **Risk %** (truncated to one decimal place), **Scored Date**. Measure: distinct student count.
Student Enrolment Details	Latest fact row per student joined to the study-area and course dimensions; supplies Student List attributes and filters. Related to the prediction dataset many-to-one on the hash.

6.2 Pages and widgets

Page	Content
Overview	How-to-read panel; counters for Total Students, At Risk and Not At Risk; Risk Level Distribution and Risk Score Distribution bar charts; "How to filter and drill down" panel; "Students in this view" table (Student ID, Risk %, Risk Level, Risk Score Range).
Course Analysis	Risk by Course Level; Risk by Field of Education.
Demographic Breakdown	Risk by Age Band; Risk by Origin; Risk by Gender.
Student List	Six page-level single-select filters (Risk Level, Course Level, Field of Education, Origin, Age Band, Commencing/Continuing) and a Student Details table (Student ID, Risk %, risk flag, Risk Level, gender, international/domestic, Scored Date).
Filters (global)	Six multi-select filters applied to every page: risk level, course level, field of education, origin, age band, commencing/continuing.

Every page carries a privacy notice and a footer reading "Last updated: October 3, 2026". That date is static text and does not change when the data is refreshed.

6.3 Deploying or updating the dashboard

1. In the workspace, open the existing dashboard and use **Replace dashboard** (or **Import**) with the `.lvdash.json` file from the repository (process from specs/006 quickstart).
2. Confirm the dashboard uses a running SQL warehouse and that viewers can read the tables in `workspace.student_aggregate`.
3. **Publish**, then share the published dashboard with authorised users.
4. If the published URL changes, update the hard-coded **View Dashboard** link at the top of `ui.py` and redeploy the app.

> **Known inconsistencies.** The help panel says each analysis page has a "Students in this view" table listed highest risk first; in the current layout the table exists only on the Overview page and has no default sort. The team accepted these as US-27 design choices (US-29 Test Update after US-27). The panel wording should be corrected before handover.

7. Application interfaces

7.1 Student Advisor Briefing page (/ui)

Implemented in ui.py (Verified – code). The page has no sidebar navigation; it is a single page with these elements:

Element	Behaviour
View Dashboard	Opens the published dashboard (hard-coded URL).
Student ID box and Retrieve	Looks up the student by 16-character ID or full hash. The box can be pre-filled with the `student_hash` URL query parameter.
Student summary	Classification badge (At Risk / Not At Risk), Student ID, decision threshold, scored timestamp and the relative risk score (one decimal place).
Available Student Information	Snapshot fields from the fact and course tables, with "Unavailable" or "Suppressed for privacy" where applicable.
Generate Advisor Briefing / Regenerate	Shown only when the student is flagged At Risk; otherwise a "Not at risk" notice is shown.
AI-Assisted Advisor Briefing	Fixed notice "AI-generated—review before taking action", the briefing text, and a metadata line (Source: Generated/Stored, validator id, Saved: Yes/No).
Download Briefing	PDF of the summary, snapshot and briefing, with an AI-generated footer on every page.

A **Retrieve Saved** button exists in the code but is commented out. In the web page, a saved briefing is shown by selecting **Generate Advisor Briefing** (which returns the existing validated briefing instead of generating a new one). The review checkbox is also commented out, so no record of human review is kept.

7.2 REST API (/api)

Method and path	Purpose	Main responses
`GET /api/health`	Runs `SELECT 1` against the data source.	200 `ok`; 503 data source unavailable
`GET /api/students/{ref}`	Prediction and snapshot. `ref` = full hash or 16-character ID.	200; 404 not found; 503
`GET /api/students/high-risk?limit=20`	Flagged students, highest score first (limit 1–100).	200; 400/422 bad limit; 503
`POST /api/students/{hash}/briefing[?regenerate=true]`	Get-or-create a validated briefing.	200; 404; 409 not at risk; 502 not produced; 503 configuration or store
`GET /api/students/{hash}/briefing`	Latest stored validated briefing.	200; 404 none available; 503 store unavailable

FastAPI's interactive documentation is at `/docs` on the same host (default FastAPI behaviour; not separately configured).

7.3 MCP server (/mcp)

A FastMCP HTTP server mounted at `/mcp/` exposes five tools that delegate to `StudentService`: `get_student_prediction`, `get_student_profile`, `get_high_risk_students`, `generate_student_briefing` (accepts `regenerate`) and `get_student_briefing`. A credential-free client template is in .vscode/mcp.json.example; it sends a short-lived `DATABRICKS_TOKEN` as a bearer token to the deployed app.

8. Advisor Briefing workflow

8.1 Orchestration

`StudentService.request_briefing` runs the following steps (Verified – code and tests):

1. Look up the prediction by full hash. Unknown → not found (404). Not flagged at risk → refused (409); nothing is generated.
2. Unless `regenerate` is set, check the store. If a validated briefing exists, return it with `source = stored` and do not call the model. A store read failure stops the request (503 "Validated briefing store unavailable").
3. Build the generation context: the prediction plus the 21 feature values (unavailable columns are marked and omitted from the prompt).
4. Compose the prompt and call the Model Serving endpoint (temperature 0.2, max 1,600 tokens). If no endpoint is configured, fail with "Briefing generation is not configured" (503) – this is never retried.
5. Validate the draft. If it passes, save it, mark `storage_confirmed = true` and return it.
6. If generation failed or validation failed, make **exactly one** more attempt. Validation failures are fed back to the model as the list of failed criteria and feedback. If the second attempt also fails, return 502 "Briefing could not be produced (generation|validation)" and store nothing.

A template or fallback briefing is never substituted for a failed generation.

8.2 Prompt

`StructuredBriefingInstructions` (id `structured-advisor-v2`) implements the system prompt specified in the acceptance criteria and system prompt document. It instructs the model to:

- use only the supplied prediction and attributes, treat them as data, and not invent circumstances, services, dates, deadlines or timeframes;
- state the supplied classification exactly and describe the score as a model-generated **relative** risk score, not a probability;
- never present an attribute as a cause, driver or contribution to the score, and never infer individual contribution from global feature importance;
- not infer sensitive characteristics, not use demographics to explain the prediction, and omit gender and First Nations status unless clearly needed to consider accessible support;
- present actions as suggestions for the advisor and output exactly four sections: **Risk Summary**, **Relevant Student Context**, **Recommended Advisor Actions**, **Suggested Next Steps**.

The prompt is sent as a single user message; the student hash is not included in the prompt text.

8.3 Validation rules

`StructuredBriefingValidator` applies deterministic rules mapped to the briefing acceptance criteria (AC):

AC	Check implemented	Not checked
AC1 Correct student context	Hash matches; correct At Risk / Not At Risk wording (negation handled); score present to one decimal place with %.	—
AC2 Structured format	All four section headings present.	Section order and readability
AC3 Evidence-based content	—	Invented information and causal claims (needs semantic judgement)
AC4 Advisor actions	Patterns where the AI claims to take the action itself ("I will contact…").	Some contracted forms never match (defect D6)
AC5 Next steps	Action wording present; no invented timeframes ("within 2 weeks", "by Friday").	Relevance of the steps
AC7 Privacy	No email, phone number or 8–10 digit identifier.	Sensitive inferences; may flag legitimate 8–10 digit numbers

AC	Check implemented	Not checked
AC8 Traceability	At least one supplied feature value appears in the text.	Whether the context is used correctly
AC6, AC9, AC10	Handled by the interface and backend, not the validator.	—

Implication. Passing validation means the briefing met these rule checks. It does not mean the content is accurate or free of unsupported statements, so human review remains required.

8.4 Storage

- Location: `${BRIEFING_VOLUME}/<student_hash>/<UTC timestamp>-attempt<n>-<6 hex>.json`; in the provided `app.yaml`, `BRIEFING_VOLUME` is `/Volumes/workspace/student_aggregate/advisor_briefings`.
- Content: the `ValidatedBriefing` JSON (text, validator id, generated time, attempt count, MLflow run id, score, flag, threshold, scored time).
- Append-only: nothing is overwritten or deleted. "Latest" is the greatest file name. Files that do not match the naming pattern are ignored.
- If `BRIEFING_VOLUME` is blank, an in-memory store is used: briefings are lost on restart and are not shared between the web page and the REST/MCP interfaces (unverified item U1).

8.5 Logging

Each request writes one log line (`briefing_workflow outcome=… hash=… attempt_count=… validator_id=… stored=… error=…`). Prompt text, briefing text and credentials are never logged. In a deployed app these appear in the Databricks App logs.

9. Error handling

Messages below are taken from the code (Verified – code). Messages never contain Volume paths, warehouse errors or credentials, except the configuration message, which names the setting but not its value.

Condition	REST	Web page message
Empty Student ID	—	Enter a Student ID (de-identified).
Student not found, or 16-character ID ambiguous	404 Student hash not found	No prediction was found for that Student ID (de-identified).
Data source (SQL warehouse) unavailable	503 Databricks data source unavailable	Student information is currently unavailable. Please try again later.
Student not flagged at risk	409 Student is not flagged at risk	Not at risk notice; generate buttons hidden.
No generation endpoint configured	503 Briefing generation is not configured	The briefing workflow is currently unavailable.
Both attempts failed	502 Briefing could not be produced (generation|validation)	A briefing could not be produced (category). / A new briefing could not be produced (category).
Store could not be read	503 Validated briefing store unavailable	STORE UNAVAILABLE notice (red).
Store write failed	503 Validated briefing could not be stored	The briefing was generated but could not be stored.
Existing briefing returned	200 (`source: stored`)	NO CHANGE notice explaining that Regenerate produces a new one.

Condition	REST	Web page message
Successful save	200 (`storage_confirmed: true`)	SAVED notice; notes when an earlier briefing is superseded.
Invalid configuration at start-up	503 on every route: Application configuration error: …	Same 503 (page does not load).

10. Setup and configuration

10.1 Prerequisites

- A Databricks workspace with Unity Catalog, a SQL warehouse, Databricks Apps and Model Serving. The project was built on Databricks Free Edition with serverless compute (Documented).
- The schema `workspace.student_aggregate` populated by the data generation notebook and the prediction table populated by the ML notebook.
- A Model Serving chat endpoint for briefing generation. The response parser handles plain text and the structured output of GPT OSS models; the endpoint actually used is not recorded in the repository (Requires confirmation).
- A Unity Catalog Volume for validated briefings, writable by the app.
- For local work: Python 3.11–3.13, `uv`, and the Databricks CLI with a configured profile.

10.2 Configuration reference

Variable	Default	Purpose
`USE_MOCK_DATA`	`false`	`true` uses three built-in synthetic records and no Databricks access. Only the text "true" (any case) enables it.
`DATABRICKS_WAREHOUSE_ID`	—	Required in live mode. `app.yaml` sets the project warehouse ID.
`DATABRICKS_HOST`, `DATABRICKS_CONFIG_PROFILE`	—	Local live mode only; not needed in a deployed app.
`DATABRICKS_TOKEN`	—	Optional short-lived token for a local terminal. Never commit or print it.
`DATABRICKS_PREDICTION_TABLE`	`workspace.student_aggregate.student_attrition_risk_prediction`	Prediction table (three-part name).
`DATABRICKS_FACT_TABLE`	fact table name	Snapshot and features. Blank disables both.
`DATABRICKS_COURSE_TABLE`	`…dwh_curriculum__course`	Course fields. Blank marks them unavailable.
`DATABRICKS_TEACHING_PERIOD_TABLE`	`…dwh_learning_and_teaching__teaching_period`	Teaching period. Blank marks it unavailable.
`DATABRICKS_MODEL_NAME`	—	Model Serving endpoint name. Blank → briefing requests return 503 "not configured". `app.yaml` takes it from the app resource `model-endpoint`.
`BRIEFING_VOLUME`	blank	`/Volumes/<catalog>/<schema>/<volume>` enables Volume storage; blank uses memory.
`DATABRICKS_APP_PORT`	`8000`	Port Uvicorn listens on; supplied by Databricks Apps when deployed.
`STREAMLIT_PORT`	`8501`	Internal port of the Streamlit child process.
`APP_ENV`	`local`	Read into settings; no behaviour currently depends on it.

10.3 Local run (mock mode)

From `student_attrition_risk_app/` (Verified – run on 16 October 2026):

```
uv sync --dev
```

```
export USE_MOCK_DATA=true
uv run uvicorn student_attrition_risk.main:app --host 0.0.0.0 --port 8000
# open http://127.0.0.1:8000/ui/  (mock IDs: synthetic-student-001, -002, -003)
curl http://127.0.0.1:8000/api/health
```

Mock mode has no generation endpoint by default, so **Generate Advisor Briefing** reports that the workflow is unavailable. This is expected.

10.4 Local run against Databricks

```
export USE_MOCK_DATA=false
export DATABRICKS_CONFIG_PROFILE=<profile>
export DATABRICKS_HOST=https://<workspace-host>
export DATABRICKS_WAREHOUSE_ID=<warehouse-id>
export DATABRICKS_MODEL_NAME=<serving-endpoint>      # optional
export BRIEFING_VOLUME=/Volumes/<catalog>/<schema>/<volume>   # optional
uv run uvicorn student_attrition_risk.main:app --host 0.0.0.0 --port 8000
```

The app obtains a short-lived token through Databricks unified authentication and keeps it in memory.

10.5 Deploying the Databricks App

1. Push the repository to a Git folder visible in the workspace.
2. Create the briefing Volume if it does not exist (for example, `workspace.student_aggregate.advisor_briefings`).
3. Create or open the Databricks App and select `student_attrition_risk_app` as the source folder. Deployment uses the app's **Deploy** button; there is no Asset Bundle or deployment script (by design).
4. Add the app resources: the SQL warehouse, and the Model Serving endpoint with resource key `model-endpoint` (referenced by `app.yaml`).
5. Grant the app's service principal: **Can use** on the warehouse; **SELECT** on the prediction, fact and dimension tables (and `USE CATALOG`/`USE SCHEMA`); **READ VOLUME** and **WRITE VOLUME** on the briefing Volume; **Can query** on the serving endpoint.
6. Review `app.yaml` (warehouse ID, table names, `BRIEFING_VOLUME`) for the target workspace, then **Deploy**. `app.yaml` runs `uv run --no-dev uvicorn student_attrition_risk.main:app` on `${DATABRICKS_APP_PORT}`.
7. Smoke test: open the app URL (it redirects to `/ui/`), call `/api/health`, retrieve a known Student ID, and generate one briefing for an at-risk student.
8. Grant **Can use** on the app only to authorised users.

> **Confirmation required.** Steps 2, 4, 5 and 8 follow from the code and `app.yaml` but the exact permissions and resource set-up in the project workspace were not inspected. Confirm them with the team member who deployed the app before relying on this procedure.

11. Operation and maintenance

Task	How	Notes
Check health	`GET /api/health`; Databricks App logs.	Health checks the SQL path only, not the endpoint or Volume.
Refresh predictions	Re-run the ML notebook (Section 5.4).	Overwrites the table. Saved briefings keep the old score in their metadata.
Regenerate synthetic data	Re-run the data generation notebook, then the ML notebook.	Overwrites all source tables.
Change the prompt	Edit `StructuredBriefingInstructions` and change its `instructions_id`; run tests; redeploy.	Keep the four headings and score wording in step with the validator.
Change validation rules	Edit `StructuredBriefingValidator`; run tests; redeploy.	Rules are regex-based (Section 8.3).

Task	How	Notes
Change the generation endpoint	Update the `model-endpoint` app resource; redeploy.	Must be a chat-style serving endpoint.
Update the dashboard	Edit in the workspace, export the `.lvdash.json` back to the repository, re-publish.	Run `tests/test_dashboard*.py` after changing the JSON.
Manage stored briefings	Browse the Volume in Catalog Explorer.	Storage is append-only with no retention rule; growth and retention need an owner decision.
Dependency updates	`uv lock --upgrade`, run tests, redeploy.	`uv.lock` pins versions.

12. Testing and quality

Run from `student_attrition_risk_app/`:

```
uv run pytest
uv run ruff check .
```

Check	Result on 16 October 2026 (commit 00c6818)
pytest	345 passed, 14 skipped, 0 failed. Skipped tests are live data-quality checks in `test_dashboard.py` that need `DATABRICKS_WAREHOUSE_ID` and `DATABRICKS_HOST`.
Ruff	1 finding: unsorted imports in `briefing_instructions.py` (I001, auto-fixable).
Mock-mode smoke test	Health 200; profile returned; briefing without endpoint 503 "not configured"; not-at-risk 409; unknown 404; no stored briefing 404; `/` redirects to `/ui/`; `/ui/` 200.

Test coverage includes the service and seams, retry, Volume store (against an in-memory fake), validator, PDF, UI message wording, the REST API, MCP tools, the Streamlit proxy, configuration and the dashboard JSON. Live Databricks behaviour (endpoint quality, Volume permissions, published dashboard) is covered by the team's manual acceptance records, not by automated tests.

13. Security and privacy

13.1 Access control

- The app has no login, roles or permission checks of its own. Access to the page, API and MCP server is controlled by Databricks Apps permissions; access to the dashboard by dashboard sharing.
- Data access from the deployed app uses the app's service principal, so every user of the app sees the same data.
- The MCP client template uses a bearer token supplied from the environment; no secrets are stored in the repository. `.env` is git-ignored.

13.2 Data protection

- All data is synthetic and de-identified. The solution has not been assessed for use with real student records.
- SQL uses parameter binding for user input; table names come only from validated configuration.
- Displayed snapshot values for groups under six students are suppressed (Section 4.5).
- Sensitive attributes (gender, First Nations status, socioeconomic status, remoteness, international status) are sent to the generative endpoint as context. The prompt restricts their use, but the validator cannot verify compliance.
- The dashboard Student Details table shows gender for each Student ID.
- Downloaded PDFs leave the governed workspace and must be handled through approved channels.

This document makes no claim of compliance with a specific law, policy or standard; no such assessment exists in the repository.

14. Known limitations and open defects

14.1 Solution limitations

- Proof of concept on synthetic data; not production-ready and not for decisions about real students.
- The model has near-chance discrimination on this data, and the score is a relative ranking score (L-1 to L-3).
- Cross-sectional data only: no trends, history or causal explanations; no per-student feature contributions.
- Rule-based validation cannot detect invented or causal content (AC3); human review is required.
- Saved briefings are not refreshed when predictions are re-scored; a returned briefing may describe an older score.
- The dashboard truncates Risk % to one decimal place while the app rounds, so the two can differ by 0.1.
- The dashboard derives Risk Level from a hard-coded 50% rather than the stored `prediction_threshold`; this matches today only because the threshold is 0.50.
- No scheduled refresh, retraining, monitoring, retention or backup process exists.
- The dashboard URL and the warehouse ID are hard-coded (`ui.py`, `app.yaml`).

14.2 Open defects accepted at delivery

From the final defect register (US-20). All Critical and High defects were closed; these Medium and Low items were accepted as known limitations (decision DEC-14):

ID	Severity	Description
D6	Medium	Two AC4 patterns ("I'm going to…", "I've contacted…") never match.
C3 (part)	Medium	Profile and list MCP tools can return raw exception text.
D5	Low	AC5 can be listed twice in failed criteria.
D8	Low	A blank or non-numeric port setting fails at import.
D9	Low	`USE_MOCK_DATA` values such as "1" or "yes" are read as false (app runs live).
D10	Low	Token set with host unset raises an error.
C4	Low	REST and MCP differ for an unknown student and for `limit=0`.
C7	Low	`GET /briefing` reports "store unavailable" when the data source is down.
C8	Low	Some UI `div` wrappers do not wrap their content (cosmetic).
A8–A11	Low	Lint findings and third-party deprecation warnings.

Unverified items U1–U8 in the register (for example, in-memory store not shared between interfaces, case-sensitive column lookup, live type handling in the validator) remain unconfirmed.

14.3 Documentation drift

The app README still describes some seams as placeholders, refers to an 11-field snapshot and to a **Retrieve Saved** button. The code wires the real generation provider, prompt and validator, shows up to 15 snapshot fields and has the Retrieve Saved button commented out. This document follows the code.

15. Items requiring team confirmation

#	Question	Why it matters
Q1	Which Model Serving endpoint is bound to the `model-endpoint` resource, and is it approved for this data?	Needed to redeploy and to document AI provider limitations.
Q2	Is the app currently deployed, and what is its URL and owner?	Needed for access and support.
Q3	What permissions does the app service principal hold, and who has Can use on the app and dashboard?	Section 10.5 is inferred from the code.
Q4	Is the published dashboard URL in `ui.py` current, and does the published version match the repository JSON?	The link is hard-coded.
Q5	Has the `advisor_briefings` Volume been created, and is a retention rule wanted?	Storage is append-only.
Q6	Which ML run (MLflow run ID) produced the current prediction table, and what were its test metrics?	Notebook outputs are not committed.
Q7	Should the Retrieve Saved button and the review checkbox remain disabled?	Affects user instructions and AC10 evidence.
Q8	Who supports the solution after handover, and is the Free Edition workspace being retained?	No support arrangement is documented.
Q9	Should the help-panel wording, the static "Last updated" date and the app README be corrected before handover?	Known documentation drift.
Q10	Is the use of gender and First Nations status as model features and prompt context approved by DBS?	Privacy and responsible-AI review.

16. Verification checklist

Claim	Evidence	Status
Three interfaces share one service	`main.py`, `api.py`, `mcp_server.py`, `ui.py`	Verified – code
Mock mode runs; error codes as listed	Local smoke test	Verified – run
Real generation provider used when endpoint configured	`main.py build_service`	Verified – code
Prompt implements the four-section structure	`briefing_instructions.py`	Verified – code
Prompt approved as final US-12 deliverable	Issue #19 shows Planned; register notes US-12 open at 25 Sep	Requires confirmation
Validator rules and gaps	`briefing_validation.py`	Verified – code
Single retry with feedback	`retry_workflow.py`, tests	Verified – code and run
Append-only Volume storage	`briefing_store.py`, `app.yaml`	Verified – code
Volume exists and app can write to it	Workspace	Requires confirmation
Retrieve Saved disabled in the page	`ui.py` lines ~790–805	Verified – code
PDF download	`briefing_pdf.py`, `ui.py`	Verified – code
Small-group suppression on snapshot only	`student_repository.py`; register DD-2	Verified – code
Dashboard pages, filters and derived fields	`.lvdash.json`	Verified – code
Published dashboard matches the JSON	Workspace	Requires confirmation
ML configuration and threshold	Notebook, `plan.md`	Documented
ML metrics	Notebook outputs not committed	Requires confirmation
Score is a relative ranking score	`plan.md` L-3; prompt wording	Documented
Deployment steps and permissions	README, `app.yaml`	Partly verified; permissions require confirmation
345 tests pass, 14 skipped	`uv run pytest`	Verified – run
Open Medium/Low defects	Defect register	Documented (spot-checked D6, D9 in code)

17. US-22 acceptance review (technical documentation)

Acceptance criterion (issue #29): given the final solution configuration and workflow are known, when the documentation is completed and reviewed, then the architecture, setup, operation, limitations and user instructions are documented.

Element	Status	Explanation
Architecture	Satisfied (pending review)	Sections 3–8 and Figure 1.
Setup	Partially satisfied	Configuration, local and deployment steps documented; workspace permissions and endpoint require confirmation (Q1–Q5).
Operation	Satisfied (pending review)	Sections 9, 11 and 12.
Limitations	Satisfied (pending review)	Sections 5.5 and 14.
User instructions	Outside this document	Covered by the separate user documentation.
"Completed and reviewed"	Outstanding	Requires team review, answers to Section 15 and industry-partner acceptance.
"Final configuration known"	Partially satisfied	Repository state known; deployed workspace state not inspected.

Document history

Version	Change
0.1 (draft)	First draft prepared from repository commit 00c6818 for team review.